Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251210**
Nama Lengkap	**Gabriel Ekklesia**
Minggu ke / Materi	**08 / Pengelolaan String dan Reguler Expression**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1

Pengantar String

String adalah kumpulan karakter yang digunakan untuk menyimpan teks dalam program komputer, seperti kata atau kalimat. Data string terdiri dari huruf, angka, atau simbol yang direpresentasikan dalam kode ASCII.

String bukan termasuk tipe data dasar karena dapat menyimpan banyak karakter dalam satu variabel. Pada beberapa bahasa pemrograman, string diperlakukan sebagai kumpulan karakter (array), seperti pada bahasa C.

MATERI 2

Pengaksesan String dan Manipulasi String

String dapat dibuat dengan sederhana menggunakan variabel:

```
namasaya = "Gabriel Ekklesia"
teman1 = "Jevino Alexandro"
teman2 = 'Dandi Putra Pratama'
teman3 = "Muhammad" + 'Nur'

print(teman3)
print(namasaya[0])   # 'G'
print(namasaya[8])   # 'E'
print(teman1[1])     # 'e'

huruf = teman2[0]
print(huruf)         # 'D'
```

MATERI 2

Operator dan metode string

Operator in

Pada string, kita dapat mengecek apakah suatu kata atau kalimat merupakan bagian dari string lain dengan menggunakan operator *in*. Operator ini akan menghasilkan nilai True atau False.

Selain itu, string juga bisa dibandingkan dengan operator perbandingan. Hasil dari perbandingan tersebut juga berupa True atau False, tergantung apakah kondisi terpenuhi atau tidak.

```
if "apel" > "anggur":
    print("Benar")
else:
    print("Salah")

if "tiga" == "tiga":
    print("Cocok")
```

Fungsi Len

Untuk mengetahui jumlah karakter dalam sebuah string, digunakan fungsi len(). Fungsi ini akan menghitung panjang string secara keseluruhan.

Untuk mengambil karakter terakhir, digunakan indeks ke len(string) - 1, karena indeks pada string dimulai dari 0.

```
kalimat = "Setiap hari adalah kesempatan untuk menjadi lebih baik"

print(len(kalimat))

terakhir = kalimat[len(kalimat)-1]
print(terakhir)

terakhir_versi2 = kalimat[-1]
print(terakhir_versi2)

terakhir2 = kalimat[-2]
print(terakhir2)
```

Traversing string

Traversing string adalah proses menampilkan isi string satu per satu (per huruf) dengan menggunakan perulangan (loop).

Traversing string dapat dilakukan dengan dua cara, yaitu menggunakan akses indeks dan tanpa menggunakan indeks secara langsung. Contoh dengan akses indeks:

```
kalimat = "tanah airku indah"
i = 0

while i < len(kalimat):
    print(kalimat[i], end='')
    i += 1
```

STRING SLICE

String slice adalah cara mengambil sebagian isi string (substring) dengan menggunakan indeks tertentu. Penulisannya menggunakan format string[awal:akhir], di mana bagian awal dimulai dari indeks 0 dan batas akhir tidak ikut diambil.

Selain itu, string bersifat immutable, artinya data string tidak dapat diubah setelah dibuat, tetapi hanya bisa diganti dengan nilai baru.

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang dinputkan	string.endswith()

startswith()	mengetahui apakah suatu string diawali dengan string yang dinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah hurufkecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depandan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Operator * dan + pada String

Pada Python, operator + dan * memiliki fungsi tambahan pada string. Operator + digunakan untuk menggabungkan dua string menjadi satu. Sedangkan operator * digunakan untuk mengulang string sebanyak jumlah tertentu.

```
kata1 = "aku"
kata2 = "belajar"
kata3 = kata1 + " " + kata2
print(kata3)   # penggabungan: aku belajar

kata4 = "halo"
print(kata4 * 3)   # halohalohalo

kata4 = "hai "
print(kata4 * 2)   # hai hai
```

MATERI 3

Parsing String

Parsing string adalah cara menelusuri string bagian demi bagian untuk mendapatkan / menemukan / mengubah bagian string yang diinginkan. "Hari bersejarah terjadi pada tanggal 10-11-2020 di kota Surabaya".Dari string di atas, diinginkan untuk mengambil tanggal, bulan, dan tahun, kemudian menyusunnya menjadi format 11/10/2020.

Kita dapat menggunakan cara parsing string sebagai berikut:

• Split string berdasarkan spasi sehingga menjadi token: "Hari", "bersejarah", "terjadi", "pada", "tanggal", "10-11-2020", "di", "kota", "Surabaya".

• Selanjutnya, cari token yang diawali dengan angka.

• Kemudian split token tersebut menggunakan pemisah '-'.

• Susun kembali menjadi format yang diinginkan.

```
kalimat = "Hari bersejarah terjadi pada tanggal 10-11-2020 di kota Surabaya"

kata = kalimat.split()

for k in kata:
    if k[0].isdigit():
        bagian = k.split('-')
        hasil = bagian[1] + "/" + bagian[0] + "/" + bagian[2]
        print(hasil)
```

MATERI 4

Pengantar Regex

Dalam pengolahan string, teknik dasar seperti pengaksesan dan manipulasi seringkali cukup rumit untuk kasus tertentu. Oleh karena itu, digunakan regular expression (regex) untuk mempermudah proses tersebut. Regex adalah pola karakter yang digunakan untuk mencari, mengganti, atau menghapus bagian tertentu dalam string. Teknik ini lebih fleksibel dibandingkan metode biasa seperti split() atau find(). Python mendukung regex melalui library re. Salah satu fungsi yang sering digunakan adalah search() untuk menemukan pola dalam string. Pada pengolahan string, terdapat berbagai teknik seperti pengaksesan dan manipulasi string. Namun, penggunaan cara biasa seringkali cukup rumit untuk kasus tertentu. Untuk mempermudah proses tersebut, dapat digunakan regular expression, yaitu teknik yang membantu pengolahan string menjadi lebih cepat dan efisien. Contoh mencari nomer hp:

```
import re

teks = "Nomor saya adalah 08123456789"

cari = re.search(r'\d+', teks)

if cari:
    print(cari.group())
```

MATERI 5

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Libary Python

Tabel 8.1: Special Character pada Python

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Dakhiri dengan	this$	Diakhiri dengan kata this

*	0 s/d tak terhingga karakter	\{}d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{}d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(saya|kamu)	saya atau kamu sebagai satu kesatuan
|	atau	\{}d|\{}s	1 digit atau 1 spasi

Tabel 8.2: Escaped Character pada Regex

Special Characters	**Kegunaan**	**Contoh**
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Tabel 8.3: Himpunan Karakter pada Regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

```python
kata1 = input("Masukkan kata pertama: ")
kata2 = input("Masukkan kata kedua: ")

kata1 = kata1.lower()
kata2 = kata2.lower()

if len(kata1) != len(kata2):
    print("Bukan anagram")
else:
    # urutkan huruf lalu bandingkan
    if sorted(kata1) == sorted(kata2):
        print("Anagram")
    else:
        print("Bukan anagram")
```

Program ini dibuat untuk mengecek apakah dua kata merupakan anagram atau bukan. Pertama, pengguna diminta memasukkan dua kata melalui input. Setelah itu, kedua kata tersebut diubah menjadi huruf kecil menggunakan fungsi lower() agar perbandingan tidak terpengaruh oleh perbedaan huruf besar dan kecil. Selanjutnya, program akan mengecek panjang dari kedua kata menggunakan len(). Jika panjangnya berbeda, maka langsung disimpulkan bahwa kedua kata tersebut bukan anagram. Namun jika panjangnya sama, program akan melanjutkan dengan mengurutkan huruf dari masing-masing kata menggunakan fungsi sorted(). Hasil pengurutan ini kemudian dibandingkan. Jika kedua hasil tersebut sama,

berarti kedua kata memiliki susunan huruf yang sama sehingga termasuk anagram. Sebaliknya, jika hasilnya berbeda, maka kedua kata tersebut bukan anagram.

SOAL 2

```
kalimat = input("Masukkan kalimat: ")
kata_dicari = input("Masukkan kata yang dicari: ")

kalimat = kalimat.lower()
kata_dicari = kata_dicari.lower()

kalimat = kalimat.replace(".", "")

kata_list = kalimat.split()
jumlah = kata_list.count(kata_dicari)

print(kata_dicari, "ada", jumlah, "buah")
```

Program tersebut berfungsi untuk menghitung jumlah kemunculan suatu kata dalam sebuah kalimat yang diberikan oleh pengguna. Pada tahap awal, program menerima dua input, yaitu kalimat dan kata yang ingin dicari. Selanjutnya, kedua input tersebut dikonversi menjadi huruf kecil menggunakan metode .lower() untuk memastikan proses pencarian tidak bersifat case-sensitive, sehingga perbedaan huruf besar dan kecil tidak memengaruhi hasil. Setelah itu, program melakukan pembersihan sederhana terhadap kalimat dengan menghapus tanda titik menggunakan metode .replace(".", ""), agar kata yang diikuti tanda baca tidak dianggap berbeda.

Kemudian, kalimat yang telah diproses dipecah menjadi daftar kata menggunakan metode .split(), sehingga setiap elemen dalam daftar merepresentasikan satu kata. Proses berikutnya adalah menghitung frekuensi kemunculan kata yang dicari dalam daftar tersebut menggunakan metode .count(). Terakhir, hasil perhitungan ditampilkan dalam bentuk output yang menunjukkan jumlah kemunculan kata tersebut dalam kalimat

SOAL 3

```python
kalimat = input("Masukkan kalimat: ")
hasil = " ".join(kalimat.split())

print("Hasil:", hasil)
```

Program ini berfungsi untuk memperbaiki format spasi dalam sebuah kalimat yang dimasukkan oleh pengguna. Pertama, pengguna diminta untuk mengetikkan sebuah kalimat. Setelah itu, kalimat tersebut diproses dengan cara dipecah menjadi beberapa kata menggunakan .split(), yang secara otomatis menghilangkan spasi berlebih di antara kata-kata. Hasil pemisahan ini kemudian digabungkan kembali menggunakan " ".join(...), sehingga setiap kata hanya dipisahkan oleh satu spasi saja. Nilai yang sudah diperbaiki tersebut disimpan dalam variabel hasil, lalu ditampilkan ke layar.

SOAL 4

```python
kalimat = input("Masukkan kalimat: ")

kata_list = kalimat.split()

terpendek = kata_list[0]
terpanjang = kata_list[0]

for kata in kata_list:
    if len(kata) < len(terpendek):
        terpendek = kata
    if len(kata) > len(terpanjang):
        terpanjang = kata

print("Terpendek:", terpendek)
print("Terpanjang:", terpanjang)
```

Program ini digunakan untuk menentukan kata dengan panjang paling pendek dan paling panjang dari sebuah kalimat yang dimasukkan oleh pengguna. Pertama, pengguna diminta untuk menginput kalimat, kemudian kalimat tersebut dipecah menjadi daftar kata menggunakan metode .split(), sehingga setiap kata dapat diproses secara terpisah.

Selanjutnya, program menginisialisasi dua variabel, yaitu terpendek dan terpanjang, dengan nilai awal berupa kata pertama dalam daftar. Hal ini dilakukan sebagai acuan awal untuk proses perbandingan. Setelah itu, program melakukan perulangan untuk setiap kata dalam daftar. Pada setiap iterasi, panjang kata dihitung menggunakan fungsi len(), lalu dibandingkan dengan panjang kata yang saat ini tersimpan sebagai terpendek dan terpanjang. Jika ditemukan kata dengan panjang lebih kecil, maka variabel terpendek diperbarui. Sebaliknya, jika ditemukan kata dengan panjang lebih besar, maka variabel terpanjang akan diperbarui.

SOAL 5

```
import re
from datetime import datetime

teks = input("Masukkan teks: ")

tanggal_list = re.findall(r'\d{4}-\d{2}-\d{2}', teks)

sekarang = datetime.now()

for tgl in tanggal_list:
    tgl_obj = datetime.strptime(tgl, "%Y-%m-%d")
    selisih = (sekarang - tgl_obj).days

    print(tgl_obj, "selisih", selisih, "hari")
```

Program ini digunakan untuk mencari tanggal dalam sebuah teks, kemudian menghitung selisih hari antara tanggal tersebut dengan waktu saat ini. Pada bagian awal, program mengimpor modul re untuk pemrosesan *regular expression* serta datetime untuk pengolahan waktu dan tanggal. Selanjutnya, pengguna diminta memasukkan sebuah teks yang mungkin mengandung tanggal.

Program kemudian menggunakan fungsi re.findall() dengan pola \d{4}-\d{2}-\d{2} untuk mengekstrak semua tanggal yang memiliki format tahun-bulan-hari (YYYY-MM-DD) dari teks tersebut. Hasil pencarian ini disimpan dalam bentuk daftar berisi string tanggal. Setelah itu, program mengambil waktu saat ini menggunakan datetime.now() sebagai acuan perhitungan.

Pada tahap berikutnya, program melakukan perulangan untuk setiap tanggal yang ditemukan. Setiap tanggal yang masih berupa string diubah menjadi objek datetime menggunakan datetime.strptime() agar dapat dihitung secara matematis. Selanjutnya, program menghitung selisih antara waktu sekarang dan tanggal tersebut dalam satuan hari. Hasil perhitungan ini kemudian ditampilkan ke layar, yang menunjukkan tanggal yang diproses beserta jumlah hari selisihnya.

SOAL 6

```python
import re
import random
import string

random.seed(1)

teks = input("Masukkan teks: ")

emails = re.findall(r'\S+@\S+', teks)

for email in emails:
    username = email.split("@")[0]

    password = ""
    for i in range(8):
        password += random.choice(string.ascii_letters + string.digits)

    print(email, "username:", username, ", password:", password)
```

Di bagian awal, ada import re, random, dan string. re dipakai buat nyari pola teks (dalam hal ini email), random buat menghasilkan karakter acak, dan string buat ngambil kumpulan huruf sama angka. Nah, random.seed(1) di situ berfungsi supaya hasil random-nya tetap sama tiap kali program dijalankan—biasanya ini dipakai biar hasilnya konsisten.

Selanjutnya, program minta input dari user lewat input("Masukkan teks: "), yang disimpan ke variabel teks. Dari teks ini, program pakai re.findall(r'\S+@\S+', teks) buat nyari semua bagian yang mirip email. Pola ini artinya: karakter apa pun selain spasi, lalu “@”, lalu karakter non-spasi lagi. Hasilnya dikumpulin dalam bentuk list emails.

Setelah itu, program nge-loop setiap email yang ditemukan. Untuk tiap email, bagian sebelum tanda “@” diambil pakai split("@")[0] dan dijadiin username. Lalu program bikin password dengan cara mengulang 8 kali (for i in range(8)), dan setiap iterasi nambah satu karakter acak yang diambil dari gabungan huruf (string.ascii_letters) dan angka (string.digits). Di akhir, program nge-print hasilnya: email asli, username hasil pemotongan, dan password acak yang sudah dibuat.

LINK GITHUB: https://github.com/gilbetjuliesa-crypto/71251210_Gabriel_Ekklesia.git